Proyecto 3: Pochoco SoC de RTL a GDSII

Arquitectura de Computadores

Integrantes:
Constanza Bustamante
Elena Durán
Mario Messen

Profesor: Jorge Gómez

RESUMEN

En este proyecto adaptamos el SoC Pochoco, que contiene el juego de reflejos desarrollado en el Proyecto 2, para llevarlo desde una implementación en FPGA iCE40 hasta un macro digital formado por celdas estándar en la tecnología GF180MCU.

Para realizar esta adaptación utilizamos LibreLane y exploramos distintas alternativas de memoria, además de analizar el área, el rendimiento temporal y la implementación física del diseño.

La implementación final utiliza una ROM de instrucciones combinacional y una RAM de datos reducida. Para la corrida de referencia, con un período de reloj de 100 ns y una utilización objetivo del core del 40 %, el flujo completó las verificaciones con DRC = 0, LVS = 0 y cero violaciones de antena. Además, se cumplieron los requisitos de timing para setup y hold, con WNS mayor o igual a cero y TNS igual a cero en los corners típico y lento.

El diseño ocupa un área de die de 0,4251 mm². Aplicando el costo de referencia de US23.000 por mm² indicado en el enunciado, se obtiene un costo equivalente estimado de US9.777. En el barrido de períodos, el menor período probado que cerró timing fue de 62 ns, correspondiente a una frecuencia aproximada de 16,1 MHz. Por otra parte, al explorar distintas utilizaciones, el área mínima con la que se logró completar el flujo fue de 0,2483 mm², utilizando un FP_CORE_UTIL del 70 %.

Finalmente, comprobamos el comportamiento funcional del diseño mediante simulaciones RTL y post-síntesis. En estas pruebas, el netlist reprodujo los resultados de la simulación RTL ciclo a ciclo para cada una de las arquitecturas evaluadas.

Todos los valores presentados en este informe provienen de las corridas realizadas con LibreLane. Los resultados se recopilaron mediante scripts/collect.py y se organizaron en results/runs.csv.

1. Del RTL de FPGA al RTL de ASIC (sección 3.1)

1.1 Módulo superior asic_top

Para adaptar el diseño a ASIC incorporamos rtl/asic_top.v, que funciona como módulo superior del macro. Su función es conectar el SoC Pochoco con las señales externas necesarias para ejecutar el juego.

A diferencia de game_top.v, que estaba preparado para conectar el sistema con los pines específicos de la Go Board, asic_top.v presenta una interfaz independiente de la FPGA. De esta manera, el diseño puede implementarse utilizando la tecnología GF180MCU sin depender de las conexiones físicas de la placa anterior.

Sus puertos corresponden a las funciones lógicas del juego, como se muestra en la siguiente tabla:

Puerto	Dir.	Función
clk	in	Reloj del sistema, T = 100 ns
rst_n	in	Reset asíncrono activo en bajo (se sincroniza dentro del SoC)
btn[3:0]	in	Botones, 1 = presionado (sincronizador de 2 FF en u_periph)
led[3:0]	out	LEDs, 1 = encendido
seg1[6:0], seg2[6:0]	out	Decenas y unidades {g,f,e,d,c,b,a}, 1 = segmento encendido
spi_sclk, spi_mosi, spi_cs_n / spi_miso	in / out	Esclavo SPI

Son terminales del macro, no pads: no tienen protección ESD ni capacidad para manejar directamente un LED o un display.

1.2 Construcciones dependientes de la FPGA

Para identificar qué elementos del Proyecto 2 necesitaban cambios, primero ejecutamos LibreLane utilizando el RTL original, sin modificaciones. Esa corrida permitió reconocer varias construcciones que dependían directamente de la FPGA iCE40 o de sus recursos internos.

Luego realizamos dos versiones intermedias, p2_step2 y p2_step3, en las que fuimos abordando estos problemas de manera progresiva. De esta forma pudimos relacionar los mensajes generados por las herramientas con las modificaciones realizadas en el RTL.

Los registros originales del proceso, incluyendo los mensajes de Verilator y Yosys, se encuentran en evidence/section_3_1/.

#	Construcción en el RTL del P2	Evidencia del flujo (archivo : línea)	Reemplazo
1	Primitiva iCE40 SB_RAM40_4K en espino_register_file.v (líneas 22 y 33)	%Error-MODMISSING: Cannot find file containing module: 'SB_RAM40_4K', en E1_p2_original_verilator-lint.log:1 y :13. El flujo se detiene en el lint	Register file de 15 × 32 flip flops con reset, x0 fijo en 0 y **lectura síncrona**, para conservar la latencia de 1 ciclo y el handshake rf_ready de la etapa ID
2	initial $readmemh(MemFile, mem) en pochoco_ram.v:36	ERROR: Can not open file 'sw/game.hex' for \$readmemh., en E2_p2_original_yosys jsonheader.log:406	El assembler genera game_program.vh, que se sintetiza como ROM combinacional (estrategia 1) o como valores de reset de los FF (estrategia 2)
3	BRAM inferida de 512 × 32 en pochoco_ram.v	GF180 no tiene BRAM. Yosys la implementa con registros: Mapping memory \u_pochoco.u_ram.mem … created 512 $dff cells … of width 32 (E4_p2_step3_yosys-synthesis.log:1476-1477). Resultado: **16.468 dffq_1** y **2.328.951 µm²** de celdas (E4_p2_step3_stat.rpt:24, :50), unas 15 veces el diseño final	Memoria dimensionada al uso real del juego (sección 2)
4	Power-on reset reg [19:0] por_cnt = 20'b0 en pochoco_soc.v:32	%Warning-PROCASSINIT: Procedural assignment to declaration with initial value: 'por_cnt', en E3_p2_step2_verilator lint.log:1. Yosys lo convierte en un atributo init (E4_…_yosys-synthesis.log:297)	Puerto externo rst_n y sincronizador de 2 FF (aserción asíncrona, liberación síncrona) en pochoco_soc.v
5	Valores iniciales de FF (contenido de la memoria y por_cnt)	**Sin mensaje**: dfflegalize los mapea a dffq_1, que no tienen valor inicial, y el atributo init se descarta en silencio	Todo estado que debe partir conocido usa reset
6	Constantes de game.s calculadas para 25 MHz	**Sin mensaje**	game.s parametrizado con CLK_HZ (sección 4.4)
7	Inversión de segmentos (displays activos en bajo	**Sin mensaje**	asic_top entrega segmentos lógicos; la

	de la Go Board)		polaridad corresponde al pad o a la placa
8	Reset de 2^{20} ciclos para esperar a que las BRAM de la iCE40 estén listas	**Sin mensaje**	Ya no es necesario: no hay BRAM y el reset viene de un pin externo

Además de los problemas detectados directamente por las herramientas, identificamos otras situaciones que podían afectar una implementación ASIC aunque no generaran necesariamente un error durante la síntesis.

En primer lugar, los valores iniciales de los registros y de la memoria que se obtenían a partir de la configuración de la FPGA no se pueden dar por garantizados en un circuito integrado. Un flip-flop que no tenga un mecanismo de reset puede comenzar en un estado desconocido. Por eso, los estados que necesitan tener un valor definido al iniciar el sistema deben contar con una inicialización implementada mediante hardware sintetizable.

También fue necesario revisar las constantes temporales del programa. En el Proyecto 2 se habían calculado utilizando un reloj de 25 MHz. Al cambiar el período objetivo a 100 ns, la frecuencia pasa a ser de 10 MHz. Si hubiéramos conservado las constantes anteriores, las esperas y los tiempos mostrados en pantalla ya no habrían correspondido a los valores esperados.

Otro aspecto que revisamos fue la polaridad de las señales de los displays. En la Go Board los segmentos se manejaban con una polaridad determinada por la placa, pero en un macro ASIC las señales deben representar la función lógica del display. La adaptación eléctrica dependerá del pad o del circuito que se conecte posteriormente.

Por último, eliminamos el retardo de encendido que se había incorporado para esperar a que las BRAM de la FPGA estuvieran listas. Como en la implementación ASIC ya no utilizamos esas memorias, mantener un contador dedicado a esa espera dejaría de tener sentido y consumiría área innecesariamente.

2. Memoria y programa del juego (sección 3.2)

2.1 Uso real del juego

Antes de definir las nuevas arquitecturas de memoria, determinamos cuánto espacio necesita realmente el programa.

La revisión del assembler muestra que game.s genera 136 instrucciones. Esta cantidad también se comprobó contando las entradas generadas en game_program.vh. Como la instrucción li se expande siempre a dos palabras, cambiar la frecuencia del reloj no modifica el tamaño del programa.

En cuanto a los datos, el análisis estático del código muestra que las instrucciones lw y sw utilizan direcciones asociadas a los periféricos MMIO. No encontramos accesos a una RAM convencional, ni uso de stack o llamadas a subrutinas que requieran reservar espacio adicional.

Para complementar esta revisión, utilizamos el testbench para contar dinámicamente los accesos a la RAM de datos durante una partida completa. El resultado fue de cero accesos, incluso considerando las diez respuestas correctas y una respuesta incorrecta.

Con estos resultados, el juego utiliza 136 palabras de instrucciones y 0 palabras de datos durante su ejecución normal. Ambas cantidades quedan dentro de los límites recomendados por el enunciado: 256 palabras de instrucciones y 64 palabras de datos.

De todas formas, decidimos reservar cuatro palabras para la RAM de datos. Esto permite disponer de un pequeño margen ante posibles modificaciones futuras del programa sin aumentar demasiado el tamaño de la memoria.

2.2 Extensión del assembler

Para poder generar las nuevas implementaciones de memoria sin escribir manualmente las instrucciones de la ROM, ampliamos el assembler desarrollado en el Proyecto 2.

La versión actual de tools/assembler.py incorpora las siguientes funcionalidades:

- .equ NOMBRE, expresión: permite definir constantes con nombre y utilizar expresiones aritméticas enteras. Esto facilita mantener los tiempos del juego en función de la frecuencia del reloj.
- li rd, expresión: permite cargar constantes mediante una pseudo-instrucción que se expande a las instrucciones lui y addi. La expansión utiliza el mismo criterio de redondeo que GNU assembler.
- -D NOMBRE=valor: permite reemplazar una constante definida mediante .equ desde la línea de comandos. Esta opción resulta especialmente útil para reducir los tiempos durante las simulaciones.
- --verilog archivo.vh: genera un archivo Verilog que contiene la cantidad de palabras del programa (GAME_WORDS) y la función combinacional game_word(idx), encargada de entregar la instrucción correspondiente a cada dirección. También se incluye el desensamblado de las instrucciones como comentario para facilitar su revisión.

Una comprobación importante fue ensamblar el programa con CLK_HZ=25000000 y comparar el resultado con el game.hex original del Proyecto 2. La salida fue idéntica bit a bit, por lo que la extensión del assembler mantiene la compatibilidad con el programa anterior.

Esta comprobación forma parte de las 21 pruebas automatizadas incluidas en tests/test_assembler.py.

2.3 Estrategias implementadas

Para cumplir con la comparación solicitada en el enunciado implementamos dos alternativas de memoria. Ambas mantienen la interfaz necesaria para conectarse con el procesador Espino, pero utilizan recursos internos diferentes.

1. ROM de instrucciones más RAM de datos reducida (MEM_ROM).

En esta alternativa separamos la memoria de instrucciones de la memoria de datos.

El módulo pochoco_rom.v incorpora el archivo game_program.vh generado automáticamente por el assembler. Como el contenido del programa es fijo, la ROM puede representarse mediante lógica combinacional. Además, registramos su salida para conservar la latencia de un ciclo que esperaba la etapa de búsqueda de instrucciones del procesador.

Por otra parte, pochoco_dram.v implementa una RAM de cuatro palabras de 32 bits mediante flip-flops, con habilitación de escritura por byte. Esta memoria queda conectada únicamente al puerto de datos, mientras que las instrucciones se leen desde la ROM. Esta separación corresponde a una arquitectura Harvard.

2. Memoria unificada implementada con flip-flops (MEM_UNIFIED_FF).

La segunda alternativa utiliza el módulo pochoco_ram_ff.v, que contiene una única memoria de 140 palabras de 32 bits: 136 corresponden al programa y las cuatro restantes se reservan para datos.

En esta implementación, el contenido inicial del programa se establece mediante los valores de reset de los flip-flops, utilizando game_word() para determinar qué valor debe tener cada palabra. Se mantienen los dos puertos de acceso, uno para búsqueda de instrucciones y otro para datos, respetando el funcionamiento esperado por el procesador del Proyecto 2.

La selección entre ambas arquitecturas se realiza mediante una definición de compilación (VERILOG_DEFINES) en las configuraciones de LibreLane correspondientes.

En las simulaciones, ambas alternativas permiten completar el juego y obtener las mismas mediciones de reacción. Para evaluar la equivalencia después de la síntesis, comparamos por separado el RTL y el netlist de cada arquitectura. Esta comparación se presenta en la sección 7.

2.4 Comparación (T = 100 ns, U = 40 %)

Métrica	ROM + RAM 4 palabras	Memoria unificada FF
Área de celdas tras síntesis (µm²)	158.579	783.243
Área de celdas tras PnR (µm²)	202.986	1.066.900
Flip-flops	974	5.327
Área del die (mm²)	0,4251 (643 × 661 µm)	2,0212 (1.413 × 1.431 µm)

Utilización final	51,4 %	54,7 %
Setup WNS tt / corner lento (ns)	67,58 / 38,05	48,76 / 0,84
Hold WNS tt / corner lento (ns)	0,37 / 0,94	0,36 / 0,91
Período mínimo que cierra / frecuencia máxima	62 ns / 16,1 MHz (barrido)	≈ 99,2 ns / ≈ 10,1 MHz (estimado, ver abajo)
DRC / LVS / redes con antena	0 / 0 / 0	0 / 0 / 2
Costo equivalente	US$ 9.777	US$ 46.488

Los resultados muestran una diferencia importante entre las dos alternativas. La arquitectura ROM utiliza considerablemente menos área y alcanza una frecuencia mayor que la memoria unificada implementada con flip-flops.

En particular, la memoria unificada ocupa aproximadamente 656.073 µm², lo que representa cerca del 84 % de su área total de síntesis. Esto se debe principalmente a los 4.480 flip-flops necesarios para almacenar las 140 palabras de 32 bits, además de la lógica de escritura y los dos multiplexores de lectura de 140 entradas y 32 bits.

En la alternativa ROM el comportamiento es distinto. Como el programa no necesita modificarse durante la ejecución, sus instrucciones pueden representarse mediante lógica combinacional constante, que Yosys y ABC simplifican durante la síntesis. Por esta razón, la ROM ocupa aproximadamente 12.409 µm². La única memoria escribible es la RAM de datos de cuatro palabras.

Los resultados también muestran que reservar una memoria demasiado grande puede aumentar significativamente el área sin aportar beneficios al funcionamiento real del juego. En este caso, el programa no realiza accesos a la RAM de datos, por lo que una memoria pequeña resulta suficiente.

Frecuencia máxima de la estrategia FF.

Las corridas completas de la arquitectura FF requieren aproximadamente una hora y media en nuestro equipo. Por este motivo, realizamos la comparación física a 100 ns y estimamos su frecuencia máxima a partir del análisis estático de timing (STA).

En este flujo, el netlist se mantiene igual al cambiar CLOCK_PERIOD, por lo que utilizamos la relación entre el período y el slack del corner lento para obtener una estimación:

T_min ≈ 100 ns − 0,84 ns ≈ 99,2 ns.

Esto equivale aproximadamente a 10,1 MHz. Para la ROM, la estimación también coincide con el resultado experimental: 100 ns − 38,05 ns ≈ 61,95 ns, muy cercano al período de 62 ns que cerró en el barrido.

La ruta crítica de la arquitectura FF atraviesa el decoder, la ALU y la lógica de acceso a memoria. En este caso termina después del multiplexor de lectura de 140 entradas, lo que agrega un retardo importante respecto de la alternativa ROM.

Violaciones de antena en la estrategia FF.

En la implementación FF quedaron dos redes con violaciones de antena después de la reparación automática. Ambas corresponden a entradas de buffers relacionados con los multiplexores de lectura, y su longitud se explica en parte por el mayor tamaño físico del die.

Aunque las razones superan el límite de la regla por un margen relativamente pequeño, siguen siendo violaciones que deben documentarse. En un proceso de fabricación, estas condiciones pueden producir acumulación de carga eléctrica y dañar el óxido de compuerta de los transistores conectados.

Para esta comparación no realizamos iteraciones adicionales de reparación, ya que la arquitectura FF no fue seleccionada como implementación final. La corrida final utiliza la alternativa ROM, que terminó con cero violaciones de antena.

3. Síntesis y trazabilidad jerárquica (sección 3.3)

Método utilizado

Para analizar el área de cada parte del diseño necesitábamos conservar la jerarquía de los módulos después de la síntesis. Por esta razón configuramos SYNTH_HIERARCHY_MODE = "keep", evitando que Yosys aplane toda la estructura.

Con esta configuración, el reporte stat.json, generado durante la síntesis, permite consultar las estadísticas de cada módulo. Además, los nombres jerárquicos de las instancias se conservan en el netlist y en el layout, lo que facilita identificar físicamente los componentes.

El script scripts/synth_report.py utiliza estas estadísticas para calcular el área de cada módulo, incluyendo sus submódulos, y contabilizar los flip-flops asociados.

También separamos el contador de ciclos que se había agregado en el Proyecto 2 y lo trasladamos al módulo pochoco_cycle_counter. De esta manera, quedó identificado como una instancia independiente (u_soc/u_periph/u_cycle_counter), lo que permite medir su contribución al área total.

La implementación ROM tiene un área total de celdas de síntesis de 158.579 μm². De ese total, aproximadamente el 45,9 % corresponde a celdas secuenciales.

3.1 Las cinco celdas más usadas

Celda	Cantidad	Qué parte del diseño la explica
dffrnq_1 (FF con reset)	971	Register file 480 bits + 64 de puertos de lectura registrados = 544; RAM de datos 160; etapa IF (PC y PC de ID) 65; contador de ciclos 64; periféricos y SPI ≈ 100; salida de la ROM 31
mux2_2	823	Habilitación de escritura de cada FF (D = we ? dato : Q): 480 en el register file y 128 en la RAM; el resto en el mux de lectura de datos del SoC y en la IF
aoi22_1	555	Árboles de los multiplexores de lectura 16:1 del register file (2 puertos × 32 bits, 384 celdas) y selección de operandos y write-back
nand2_1	529	Lógica de la ROM (decodificación de 136 direcciones y bits del programa), ALU y LSU
nor2_1	300	ROM, comparaciones de la ALU y decodificación MMIO

3.2 Área por módulo

Módulo (instancia)	Área µm²	%	FF
espino_core (u_core)	104.057	65,6	611
— espino_register_file (u_rf)	**69.875**	**44,1**	544
— espino_id_stage (u_id), incluye decoder y ALU	22.389	14,1	1
— — espino_alu (u_alu)	10.153	6,4	0
— — espino_decoder	2.215	1,4	0
— espino_if_stage (u_if)	8.511	5,4	65
— espino_load_store_unit	3.212	2,0	1
pochoco_dram (u_dram, 4 palabras)	18.690	11,8	160
pochoco_periph (u_periph)	14.552	9,2	116
— pochoco_cycle_counter (u_cycle_counter)	**7.325**	**4,62**	64
pochoco_rom (u_rom, 136 palabras)	12.409	7,8	31
pochoco_spi_slave (u_spi)	6.276	4,0	52

El módulo que más área aporta es el register file (44,1 %). El contador de ciclos del Proyecto 2 ocupa 7.325 µm², el 4,62 % del área: son 64 FF (contador libre de 32 bits y marca de tiempo de 32 bits) más un incrementador de 32 bits.

3.3 Comparación con lo esperado

Antes de realizar la síntesis pensábamos que los módulos de mayor tamaño serían la ALU y la ROM. La ALU parecía una candidata importante por las operaciones aritméticas y comparaciones de 32 bits, mientras que la ROM debía contener las 136 instrucciones del programa.

Sin embargo, los resultados mostraron que el register file es el componente individual que más área ocupa. Esto se explica por la forma en que se implementa su almacenamiento: al utilizar flip-flops en lugar de una BRAM, cada bit requiere un elemento de almacenamiento y lógica adicional para controlar su escritura y lectura.

En la FPGA del Proyecto 2, el register file podía aprovechar recursos de memoria propios de la placa. En GF180MCU esos recursos no están disponibles de la misma manera, por lo que el costo de implementarlo con celdas estándar es considerablemente mayor.

La ROM, en cambio, ocupa una proporción mucho menor del área total. Aunque conceptualmente contiene 4.352 bits, su contenido no cambia durante la ejecución. Por eso la síntesis puede simplificar la lógica que representa las instrucciones, aprovechando los campos y bits constantes que se repiten entre palabras.

También observamos que la RAM de datos de cuatro palabras ocupa más área que la ROM completa. Esto

ocurre porque cada bit de una memoria escribible necesita almacenamiento y lógica de selección, mientras que el contenido de la ROM puede simplificarse como una función lógica constante.

Finalmente, reducir la RAM de datos de 16 a cuatro palabras permitió disminuir el área de celdas aproximadamente un 23 %. Este resultado refuerza la importancia de dimensionar la memoria de acuerdo con el uso real del programa.

4. Timing (sección 3.4)

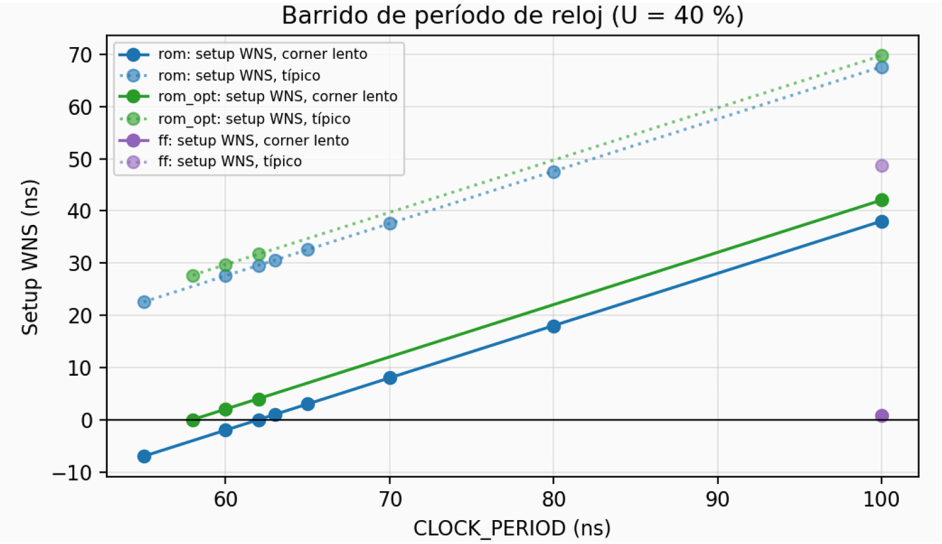
Para el análisis temporal utilizamos las restricciones SDC configuradas por defecto en LibreLane, que consideran una incertidumbre de reloj de 0,25 ns, un derate del 5 % y un límite de fanout de 10.

El análisis se realizó considerando dos corners: el típico (nom_tt_025C_5v00) y el más lento (max_ss_125C_4v50). Este último representa condiciones de proceso y temperatura más exigentes para el retardo de las celdas.

También configuramos los checkers de LibreLane para que las violaciones de setup y hold en ambos corners se consideraran al determinar si una corrida cumplía los requisitos del proyecto.

4.1 Barrido de CLOCK_PERIOD (estrategia ROM, U = 40 %)

T (ns)	Setup WNS / TNS típico	Hold WNS / TNS típico	Setup WNS / TNS lento	Hold WNS / TNS lento	Área de celdas (µm²)	¿Cierra?
100	67,58 / 0	0,37 / 0	38,05 / 0	0,94 / 0	202.986	Sí
80	47,58 / 0	0,37 / 0	18,05 / 0	0,94 / 0	202.986	Sí
70	37,58 / 0	0,37 / 0	8,05 / 0	0,94 / 0	202.986	Sí
65	32,58 / 0	0,37 / 0	3,05 / 0	0,94 / 0	202.986	Sí
63	30,58 / 0	0,37 / 0	1,05 / 0	0,94 / 0	202.986	Sí
62	**29,58 / 0**	**0,37 / 0**	**0,05 / 0**	**0,94 / 0**	**202.986**	**Sí (mínimo)**
60	27,58 / 0	0,37 / 0	−1,95 / −16,67	0,94 / 0	202.986	No
55	22,58 / 0	0,37 / 0	−6,95 / −792,57	0,94 / 0	202.986	No



En esta prueba ejecutamos el flujo con distintos valores de CLOCK_PERIOD, manteniendo la utilización objetivo en 40 %. Para cada corrida registramos WNS y TNS de setup y hold en el corner típico y en el más lento, junto con el área de celdas obtenida.

Los resultados muestran que el corner lento es el que limita la frecuencia del diseño. Aunque el circuito dispone de una holgura considerable en el corner típico, esa holgura disminuye al evaluar las condiciones más desfavorables.

El menor período probado que logró cerrar timing fue de 62 ns, equivalente a una frecuencia aproximada de 16,1 MHz. En esa corrida el setup del corner lento tiene un WNS de 0,05 ns y un TNS de cero. Al reducir el período a 60 ns, el WNS pasa a ser negativo y el TNS también presenta violaciones, por lo que esa configuración ya no cumple los criterios de cierre.

En cuanto al hold, los resultados se mantienen positivos durante el barrido. Esto se debe a que su análisis depende de los tiempos mínimos de propagación entre flancos, y no presenta el mismo comportamiento que el análisis de setup al reducir el período.

Un aspecto que observamos es que el área de celdas se mantuvo en 202.986 µm² en las corridas entre 55 y 100 ns. En este caso, cambiar el período no provocó modificaciones en la síntesis ni en las celdas ubicadas. Esto se relaciona con el uso de SYNTH_STRATEGY = AREA 0, que prioriza minimizar el área y no utiliza directamente el período de reloj para seleccionar la implementación.

Además, la reparación de timing posterior al CTS se realizó considerando el corner típico, donde todavía existía holgura. Por eso, las violaciones del corner lento se identificaron durante el análisis estático final, sin que la herramienta hubiera realizado cambios adicionales para corregirlas.

Para estudiar este comportamiento también ejecutamos una corrida a 58 ns utilizando PL_RESIZER_SETUP_SLACK_MARGIN = 10. Aunque el área cambió ligeramente, el slack del corner lento continuó siendo negativo. Esto muestra que aumentar el margen de reparación no garantiza resolver las violaciones de un corner que no está siendo utilizado como referencia principal por esa etapa del flujo.

4.2 Ruta crítica

En las corridas realizadas identificamos que la ruta crítica de setup sigue el mismo recorrido dentro del procesador y sus periféricos.

El camino comienza en un flip-flop asociado al registro de salida de la ROM, que contiene la instrucción recién obtenida durante la etapa de fetch. Desde allí, la señal atraviesa el decoder, la selección de operandos y la ALU, donde se calcula la dirección efectiva de una operación de lectura o escritura.

Luego, el resultado llega a la lógica de decodificación de direcciones MMIO del SoC y finalmente habilita el registro de lectura correspondiente en los periféricos.

Por lo tanto, aunque la ruta comienza en la memoria de instrucciones, termina dentro de pochoco_periph. No corresponde únicamente a una operación interna de la ALU, sino al recorrido completo necesario para calcular una dirección y utilizarla para acceder a un periférico.

En términos microarquitectónicos, esta ruta corresponde principalmente al acceso a memoria. Sin embargo, incluye parte del trabajo de decode y ejecución, ya que en Espino esas operaciones se realizan dentro de una misma etapa combinacional.

El retardo registrado en el corner lento fue de 61,95 ns, mientras que en el corner típico fue de

aproximadamente 32,4 ns. Esta diferencia explica por qué el corner lento determina el período mínimo que puede utilizarse.

4.3 Cambio al RTL para acortar la ruta

A partir del análisis anterior identificamos que parte del retardo se encontraba en la habilitación del registro de lectura de los periféricos.

Originalmente, este registro se actualizaba únicamente cuando se cumplía la condición access & ~we. El problema es que la señal access dependía de la dirección completa, incluyendo el bit 31, que era precisamente uno de los últimos bits en estabilizarse después del cálculo de la ALU.

Para reducir ese recorrido implementamos la optimización OPT_PERIPH_READ en rtl/pochoco_periph.v.

La idea fue aprovechar que el procesador utiliza el dato de lectura en el ciclo siguiente y que el multiplexor rd_is_per del SoC ya se encarga de seleccionar la fuente correspondiente. Por lo tanto, el registro de lectura de los periféricos puede actualizarse en todos los ciclos utilizando los bits bajos de la dirección, sin esperar a que se complete toda la decodificación MMIO.

Con este cambio eliminamos de la ruta crítica la dependencia de la condición access & ~we y parte de la lógica asociada a la decodificación de la dirección.

Métrica	rom (original)	rom_opt (con el cambio)	Diferencia
Setup WNS lento a T = 100 ns	38,05 ns	**42,07 ns**	+4,02 ns
Setup WNS típico a T = 100 ns	67,58 ns	69,72 ns	+2,13 ns
Setup WNS lento a T = 62 ns	0,05 ns	4,07 ns	+4,02 ns
Setup WNS lento a T = 60 ns	−1,95 ns (no cierra)	**2,07 ns (cierra)**	
Hold WNS típico / lento	0,37 / 0,94 ns	0,40 / 0,99 ns	
Área de celdas tras PnR	202.986 µm²	201.752 µm²	−0,6 %
Área del die	0,4251 mm²	0,4231 mm²	−0,5 %
Flip-flops	974	974	0
Violaciones de ruteo en la iteración 0	84	3	
Período mínimo que cierra	62 ns (16,1 MHz)	**58 ns (17,2 MHz)**: a 58 ns, WNS lento = +0,07, TNS = 0	−4 ns

Los resultados muestran una mejora del slack de setup en el corner lento de aproximadamente 4 ns a 100 ns. Además, el área de celdas después de PnR disminuyó cerca de un 0,6 %, por lo que la modificación mejoró el timing sin aumentar el tamaño del diseño.

La mejora, sin embargo, no fue igual al retardo total que se esperaba eliminar. Esto ocurre porque apareció una nueva ruta crítica de duración similar, asociada a la captura de la marca de tiempo del contador de ciclos cuando se escribe en el registro de los LED.

Esa operación también necesita conocer la dirección del periférico y habilitar el registro correspondiente. Por lo tanto, aunque acortamos la ruta de lectura, permaneció un camino de escritura que todavía depende de la decodificación de la dirección.

Una posible modificación adicional sería registrar la decodificación de esa dirección, pero eso introduciría un ciclo extra de latencia en las escrituras MMIO. No implementamos ese cambio porque alteraría el número de ciclos medidos por el juego y, por lo tanto, podría modificar su comportamiento temporal.

En consecuencia, dejamos la optimización actual como una mejora del camino de lectura, sin modificar la latencia funcional de las escrituras que utiliza el programa.

4.4 Constantes de game.s para T = 100 ns

Al cambiar el período del reloj a 100 ns, la frecuencia del sistema pasa a ser de 10 MHz. Esto significa que un segundo corresponde ahora a diez millones de ciclos, en lugar de los 25 millones utilizados en el Proyecto 2.

Por este motivo, fue necesario adaptar las constantes temporales de game.s. En vez de mantener los valores escritos manualmente mediante pares lui/addi, ahora las cantidades se calculan a partir de la constante CLK_HZ.

Constante	Expresión	P2 (25 MHz)	P3 (10 MHz)	lui / addi generados
CYC_TENTH (0,1 s)	CLK_HZ/10	2.500.000	**1.000.000**	0xF4 / 576
CYC_HALF_TENTH (0,05 s, redondeo)	CYC_TENTH/2	1.250.000	**500.000**	0x7A / 288
CYC_PREP (3 s de espera)	3*CLK_HZ	75.000.000	**30.000.000**	0x1C9C / 896
CYC_HALF_SECOND (0,5 s, pausa final y medio divisor)	CLK_HZ/2	12.500.000	**5.000.000**	0x4C5 / -1216
CYC_AVG_DIV (10 rondas × 0,1 s)	10*CYC_TENTH	25.000.000	**10.000.000**	0x989 / 1664
CYC_SUM_LIMIT (suma que promedia 10,0 s)	100*CYC_AVG_DIV	2.500.000.000	**1.000.000.000**	0x3B9AD / -1536

Por ejemplo, para mantener la espera de preparación de tres segundos se necesitan 30.000.000 ciclos:

30.000.000 = 3 × 10.000.000.

El assembler convierte ese valor en las instrucciones lui y addi que corresponden. Para ello separa los bits superiores e inferiores de la constante y considera que el inmediato de addi tiene signo. Cuando los 12 bits inferiores son mayores o iguales a 0x800, debe utilizarse un ajuste negativo y aumentar en uno el valor cargado

mediante lui.

De esta forma, el programa mantiene las mismas duraciones reales a pesar del cambio de frecuencia.

Después de modificar las constantes, regeneramos sw/build/game.hex y game_program.vh. Las instrucciones que cambian respecto del Proyecto 2 corresponden a los seis pares lui/addi utilizados para construir los valores temporales.

Finalmente, verificamos que el contador de 32 bits no se desborde durante el intervalo máximo medido por el juego. A 10 MHz, el desbordamiento ocurre aproximadamente cada 429 segundos, por lo que queda muy por encima del límite de diez segundos que considera el programa.

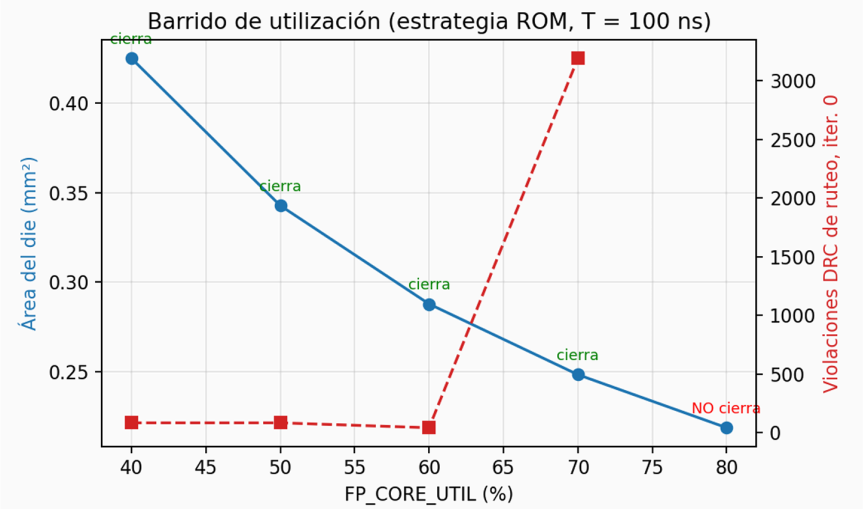
5. Floorplan, alimentación y placement (sección 3.5)

5.1 Barrido de utilización (ROM, T = 100 ns)

Para estudiar cómo influye la utilización del core en el tamaño físico del diseño, ejecutamos varias corridas manteniendo constante el período de reloj en 100 ns y modificando el valor de FP_CORE_UTIL.

La corrida final utiliza el 40 %, tal como establece el enunciado. Sin embargo, también probamos utilizaciones del 50 %, 60 %, 70 % y 80 % para observar cómo cambia el área y qué efecto tiene sobre la capacidad del flujo para ubicar y conectar las celdas.

FP_CORE_UTIL	Die (µm)	Die (mm²)	Utilización final	Violaciones de ruteo iter. 0	DRC / LVS / antena	Setup WNS lento	Largo de cables (µm)	¿Cierra?
40 %	643 × 661	0,4251	51,4 %	84	0 / 0 / 0	38,05	409.272	Sí
50 %	577 × 595	0,3428	63,5 %	84	0 / 0 / 0	44,51	374.990	Sí
60 %	528 × 545	0,2878	75,2 %	42	0 / 0 / 0	44,47	382.491	Sí
70 %	**489 × 507**	**0,2483**	**87,4 %**	**3.189**	**0 / 0 / 0**	**46,87**	**485.839**	**Sí (área mínima)**
80 %		0,2186	—	—	—	—	—	No: falla el placement detallado (DPL 0036)



Los resultados muestran que al aumentar la utilización disminuye el área del die, ya que se reduce el espacio libre disponible dentro del core. Sin embargo, esto también hace que las celdas queden más juntas y que el ruteo tenga menos alternativas para conectar las señales.

Con una utilización del 70 % conseguimos completar el flujo con un die de 0,2483 mm², que corresponde al área mínima entre las configuraciones evaluadas que lograron finalizar correctamente.

Al probar el 80 %, el placement detallado falló porque ya no había suficiente espacio en las filas para ubicar y legalizar todas las celdas necesarias. Esto incluye no solo la lógica del diseño, sino también los buffers, celdas de hold, tap cells y otros elementos agregados durante el flujo.

También observamos que la congestión no aumenta de manera gradual. Hasta el 60 % la cantidad de conflictos de ruteo de la primera iteración se mantiene relativamente baja, pero al llegar al 70 % aumenta considerablemente. Aun así, el router logra resolver los conflictos y la corrida termina sin violaciones finales.

Por lo tanto, el 70 % representa el límite práctico entre las configuraciones probadas: permite reducir el área, pero ya genera una congestión importante. La utilización del 80 % no logra completar el placement.

5.2 Red de alimentación

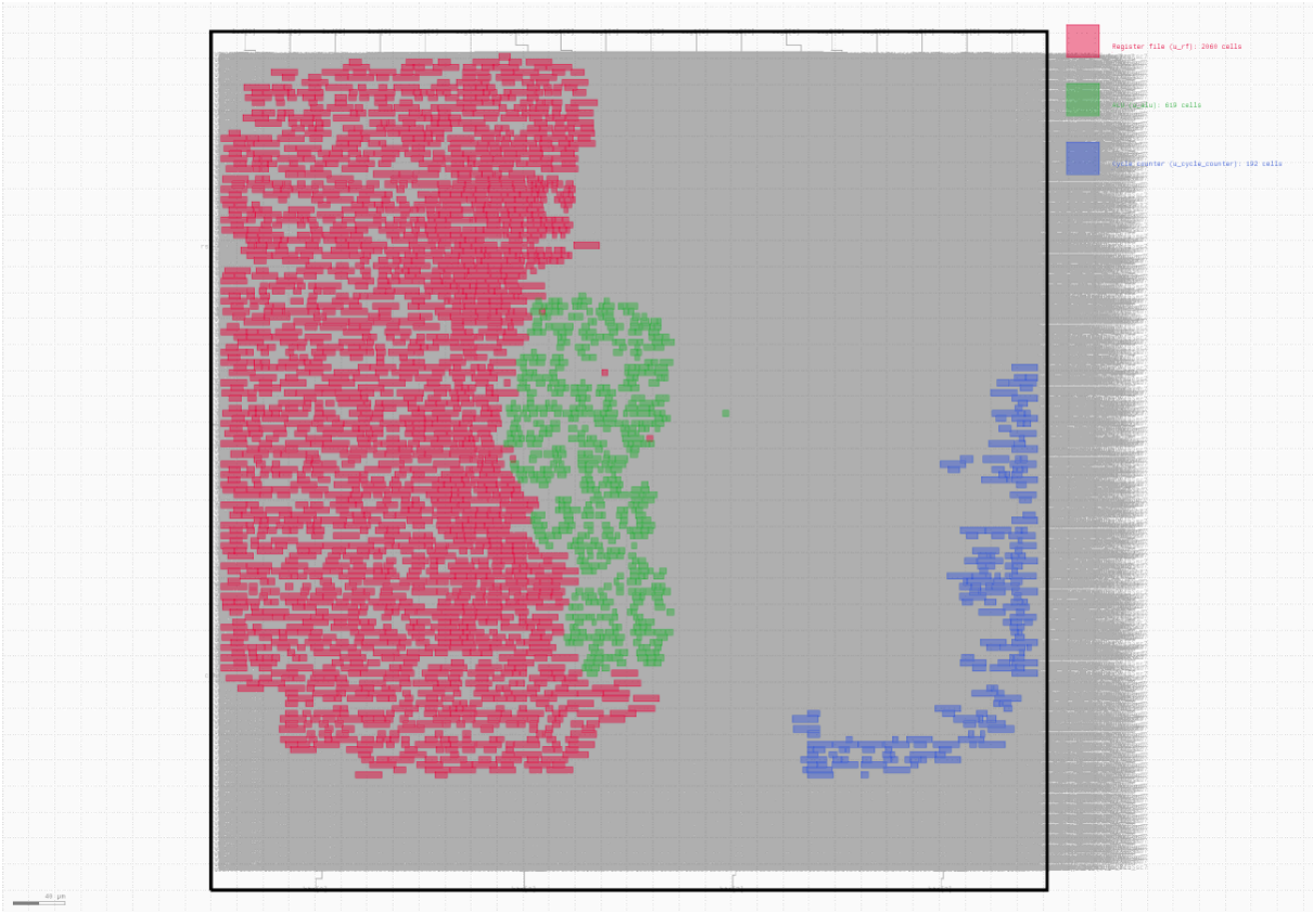
Para distribuir la alimentación entre las celdas estándar, LibreLane generó la red PDN utilizando la configuración prevista para macros en GF180MCU.

En esta implementación no se agregó un anillo propio de alimentación, ya que el macro está pensado para conectarse a la red de alimentación de un chip mayor. En su lugar, se utilizaron straps verticales de VDD y VSS en Metal4 y conexiones horizontales en Metal5. Las celdas estándar reciben alimentación mediante los rieles ubicados en Metal1.

También se incorporaron tap cells a intervalos de 20 µm para mantener la polarización adecuada del pozo y del sustrato, además de endcaps en los extremos de las filas.

El chequeo de la red de alimentación no encontró violaciones en las conexiones VDD ni VSS. Esto indica que la distribución generada cumple los criterios comprobados por el flujo.

5.3 Ubicación de los terminales



Borde	Terminales	Justificación
Oeste	clk, rst_n	Las señales globales entran juntas por un lado: el árbol de reloj parte desde ahí y quedan lejos de las salidas que conmutan
Sur	btn[3:0]	Entradas del usuario, agrupadas junto a su sincronizador
Norte	led[3:0], seg1[6:0], seg2[6:0]	Las 18 salidas visibles quedan juntas y se pueden rutear al mismo grupo de pads de salida
Este	spi_sclk, spi_mosi, spi_cs_n, spi_miso	El bus SPI completo en un solo borde, para minimizar el sesgo entre señales del bus

La ubicación de los terminales se definió agrupando las señales según su función. La idea fue mantener juntas las entradas relacionadas y evitar distribuir señales de un mismo grupo por distintos lados del macro sin necesidad.

Por ejemplo, el reloj y el reset se ubicaron en el borde oeste, mientras que los botones quedaron en el borde sur, cerca de la lógica que los sincroniza. Las salidas de los LED y los displays se agruparon en el borde norte, y

las señales SPI se ubicaron en el borde este para mantener el bus concentrado en una misma zona.

Esta distribución busca facilitar las conexiones del macro con los circuitos que podrían rodearlo en un chip completo.

Diferencia entre terminales del macro y pads de I/O.

Los terminales que aparecen en el layout son conexiones metálicas del bloque digital. No tienen por sí solos las características eléctricas necesarias para conectarse directamente a componentes externos.

En un chip completo sería necesario incorporar pads de entrada y salida, protección ESD, drivers capaces de manejar las corrientes de los LED y displays, y circuitos de adaptación eléctrica cuando corresponda. También se necesitarían las conexiones de alimentación y el espacio destinado al encapsulado.

Por lo tanto, el layout obtenido representa el macro digital solicitado en este proyecto, no un chip completo listo para conectar directamente a una placa.

5.4 Costo estimado

Con el supuesto lineal de US$ 23.000 por mm²:

Corrida final (ROM, U = 40 %): die de 643,08 × 661,00 µm = **0,4251 mm²**, costo ≈ 0,4251 × 23.000 = **US$ 9.777**.
Área mínima que completa el flujo (ROM, U = 70 %): 489,41 × 507,33 µm = **0,2483 mm²**, costo ≈ 0,2483 × 23.000 = **US$ 5.711**.
Memoria unificada de FF (U = 40 %): 1.412,77 × 1.430,69 µm = **2,0212 mm²**, costo ≈ 2,0212 × 23.000 = **US$ 46.488**.

La estimación permite comparar las alternativas de diseño, pero no debe interpretarse como el precio real de fabricar un circuito integrado.

En una fabricación real existen otros factores que influyen en el costo, como el costo fijo de las máscaras, los gastos de ingeniería no recurrentes (NRE), el rendimiento de fabricación, los pads de entrada y salida, el encapsulado y las pruebas eléctricas. Además, el macro de este proyecto no incluye todos los elementos necesarios para formar un chip completo.

Al comparar las decisiones de diseño, la que más influye en el área y en el costo estimado es la arquitectura de memoria. La memoria unificada con flip-flops requiere muchos más elementos de almacenamiento que la alternativa ROM, mientras que el programa del juego permanece fijo y no necesita modificarse durante la ejecución.

También comprobamos que reducir la RAM de datos de 16 a cuatro palabras disminuye el área de celdas, y que aumentar la utilización del core permite reducir el área total del die, aunque a costa de una mayor congestión.

Por último, la optimización de la ruta crítica tuvo un efecto pequeño sobre el área, pero permitió mejorar el slack de setup. Esto muestra que las decisiones de arquitectura y memoria tuvieron un impacto mayor sobre el tamaño del diseño que la optimización temporal realizada.

6. Signoff y layout (sección 3.6)

6.1 Resultados finales

Chequeo	Herramienta	Resultado (corrida final)
DRC	Magic (magic__drc_error__count)	**0**
DRC de ruteo	OpenROAD TritonRoute (route__drc_errors)	**0**
LVS	Netgen (design__lvs_error__count)	**0** (0 dispositivos, redes y pines sin correspondencia)
Antena	OpenROAD check_antennas (antenna__violating__nets)	**0** redes y 0 pines
XOR Magic vs. KLayout	KLayout	0 diferencias
Grilla de alimentación	PowerGridViolations	0

Los resultados de la corrida final ROM fueron satisfactorios: no se encontraron violaciones DRC, el LVS no detectó diferencias de conectividad y el chequeo de antena terminó sin redes ni pines problemáticos. También se obtuvo un resultado limpio en la verificación de la red de alimentación.

Durante el análisis aparecieron advertencias relacionadas con max slew y max cap en algunos corners lentos. Estas advertencias corresponden a restricciones eléctricas de diseño y no se encuentran entre los criterios específicos de cierre de timing establecidos en el enunciado.

En particular, varias señales de selección del register file presentan un fanout elevado, ya que una misma señal controla numerosos elementos de los multiplexores de lectura. Esto provoca transiciones más lentas en algunas redes.

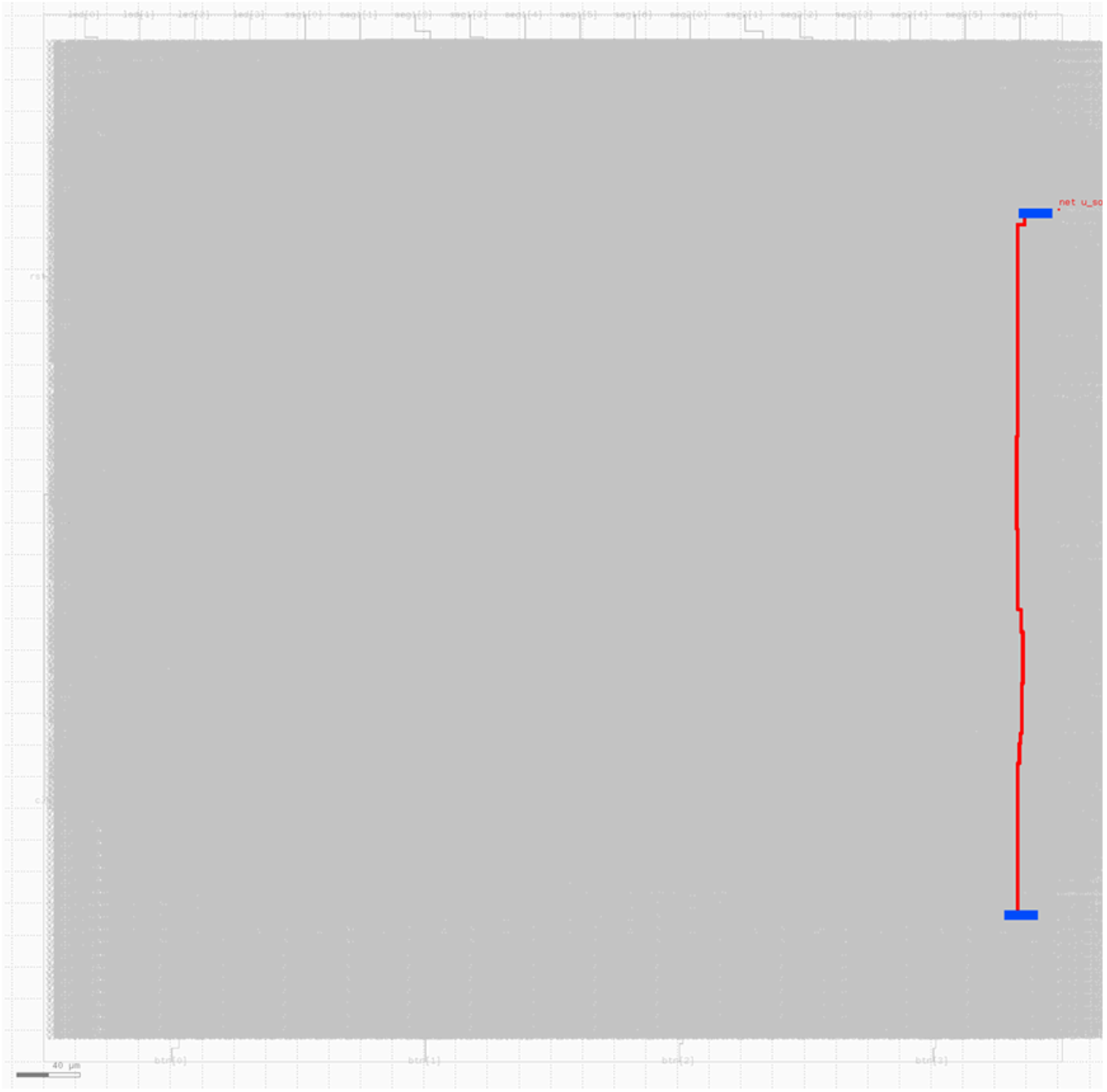
Aunque estas condiciones no invalidan los resultados de setup y hold reportados, sí son importantes desde el punto de vista eléctrico, porque una transición lenta puede aumentar el retardo de las celdas conectadas y el consumo de potencia. Por este motivo, dejamos documentados los valores encontrados y la razón por la que aparecen como advertencias y no como errores que impidan cerrar el flujo.

6.2 Violación de antena durante el desarrollo

Durante el desarrollo también encontramos un problema relacionado con las reglas de antena. Después del ruteo global, OpenROAD detectó 14 redes que superaban el límite permitido.

La violación más importante correspondía a la señal spi_sclk, específicamente al primer sincronizador de la entrada SPI. Los dos flip-flops del sincronizador habían quedado separados por una distancia considerable dentro del macro, lo que produjo un tramo largo de metal conectado a la entrada de uno de ellos.

El problema de antena ocurre porque, durante algunas etapas de fabricación, las pistas metálicas pueden acumular carga eléctrica antes de que exista una conexión adecuada para descargarla. Si esta carga se transmite hacia la compuerta de un transistor, existe riesgo de dañar su óxido.



Para resolver las violaciones, LibreLane ejecutó el paso de reparación de antena. Primero se insertaron jumpers, que permiten utilizar capas metálicas superiores para reducir la longitud de los tramos conectados a las compuertas durante ciertas etapas de fabricación.

Después se insertaron diodos de antena en las redes que todavía presentaban problemas. Estos diodos proporcionan un camino de descarga hacia el sustrato.

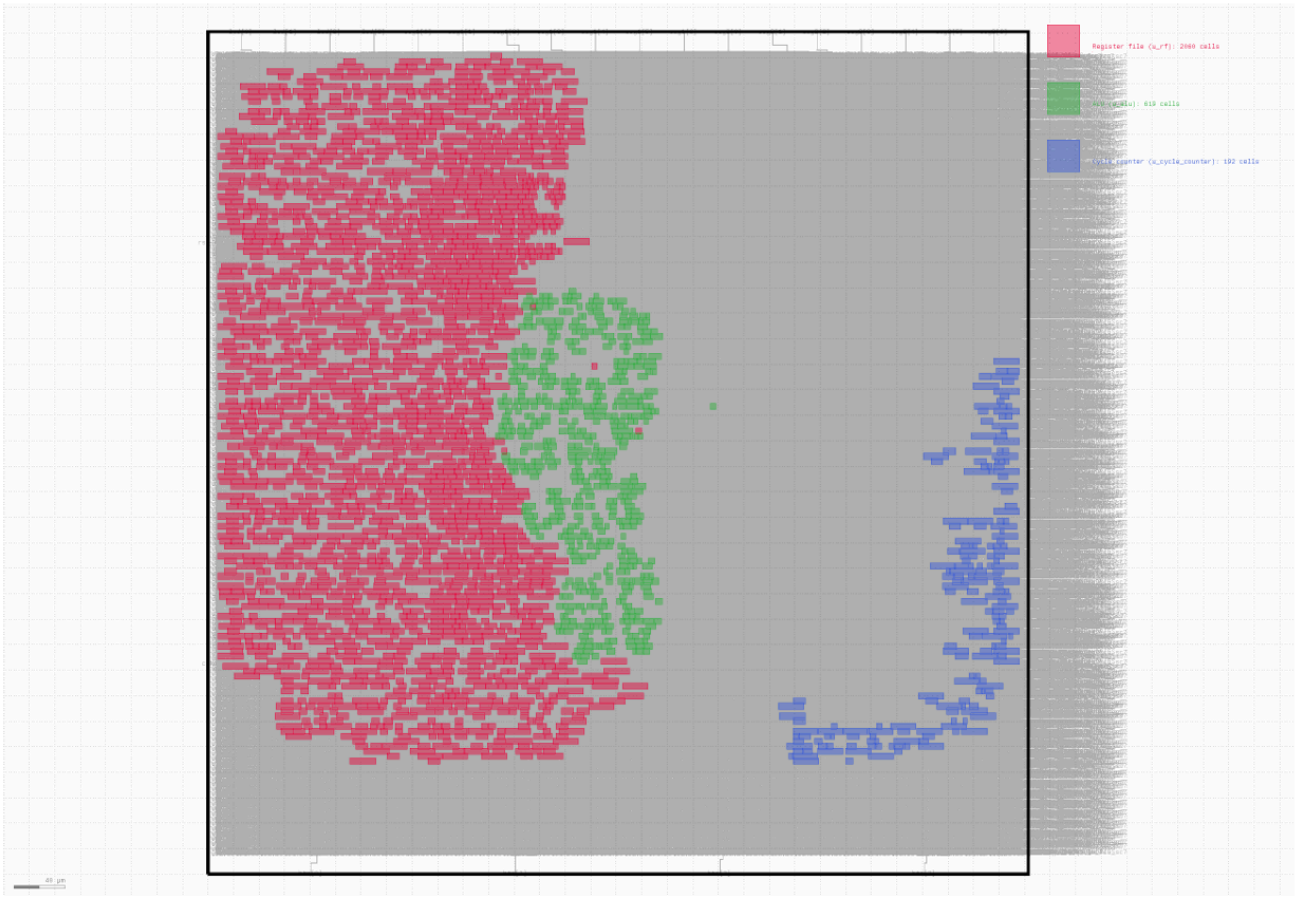
Una vez finalizado el ruteo detallado, volvimos a ejecutar la verificación. El reporte final indicó cero violaciones tanto en redes como en pines, por lo que el problema quedó resuelto para la corrida final.

Una posible mejora a nivel de distribución física sería ubicar las etapas del sincronizador SPI más cerca de sus terminales de entrada. De esta manera se reduciría desde el principio la longitud de las conexiones entre los flip-flops.

6.3 Layout anotado

Para identificar físicamente los componentes principales del SoC utilizamos scripts/annotate_layout.py, que trabaja sobre el DEF final y utiliza los nombres jerárquicos conservados durante la síntesis.

El script busca las celdas asociadas a cada instancia, las identifica y genera una representación anotada sobre el GDS. De esta manera podemos relacionar los módulos que aparecen en los reportes de síntesis con su ubicación dentro del layout.



Bloque	Prefijo de instancia	Celdas	Ubicación (µm)
Register file	u_soc/u_core/u_rf/	2.060	(8, 86) – (345, 644): mitad izquierda
ALU	u_soc/u_core/u_id/u_alu/	619	(225, 165) – (398, 460): centro, junto al register file
Contador de ciclos	u_soc/u_periph/u_cycle_counter/	192	(447, 86) – (636, 405): borde este

Como se observa en el resultado, el register file ocupa una zona importante del lado izquierdo del macro. La ALU quedó ubicada cerca de él, lo que tiene sentido porque recibe los operandos que salen de sus puertos de lectura.

El contador de ciclos, por su parte, quedó en una zona más cercana al borde este, junto con otros periféricos. Esta distribución permite visualizar cómo el placer organizó los bloques según las conexiones del diseño.

7. Simulación funcional post-síntesis (sección 3.7)

Configuración de la simulación

Para comprobar que la síntesis no modificó el funcionamiento del juego realizamos una simulación del netlist generado por LibreLane y la comparamos con la simulación RTL.

Como el programa original incluye esperas de varios segundos, utilizamos una configuración temporal reducida para que las pruebas pudieran terminar rápidamente. Para ello ensamblamos game.s con -D CLK_HZ=2000, de modo que cada décima de segundo equivale a 200 ciclos y la espera de preparación de tres segundos equivale a 6.000 ciclos.

Este cambio solo se utiliza en la simulación. Tanto el RTL como el netlist se ejecutan con el mismo programa generado en sw/build_sim/, por lo que las condiciones de comparación se mantienen iguales.

El script scripts/gate_sim.sh se encarga de preparar la corrida, sintetizar asic_top con LibreLane y ejecutar las dos simulaciones: una sobre el RTL y otra sobre el netlist post-síntesis. Para esta última se utilizan los modelos funcionales de las celdas GF180, sin retardos SDF anotados.

Estímulos aplicados en ciclos definidos

El testbench tb/tb_game.v genera las pulsaciones de los botones en momentos determinados del juego. Para que la comparación sea reproducible, los estímulos se aplican en el flanco de bajada y se cuentan los ciclos desde el instante en que aparece el LED objetivo.

Las diez pulsaciones correctas se programaron en los siguientes ciclos relativos: +250, +333, +460, +180, +610, +299, +777, +405, +150 y +520. Cada botón se mantiene presionado durante 300 ciclos.

Además, durante la tercera ronda se introduce primero una pulsación incorrecta en el ciclo +120. Esto permite comprobar que el juego detecta el error, muestra EE y reinicia la ronda antes de aceptar la respuesta correcta.

El testbench registra los cambios de los puertos de salida y los tiempos de reacción medidos por el programa, almacenados en el registro x15.

Resultado de la estrategia ROM

```
[cycle 6654] round 1: target=0010 press@+250, measured=269 cycles, display=01 [cycle 13360]
round 2: target=0010 press@+333, measured=353 cycles, display=02 [cycle 19869] round 3:
WRONG button 0010 at +120 -> display EE, round restarted (LEDs=1111) [cycle 26686] round 3:
target=0010 press@+460, measured=482 cycles, display=02 ...
[cycle 74281] round 10: target=0100 press@+520, measured=538 cycles, display=03
[cycle 75134] average display=02, LEDs=0000
data RAM accesses during the whole game: 0
RESULT: PASS (errors=0)
=== Comparing output-port and measured-cycle logs
RTL and netlist logs are IDENTICAL (47 events)
```

La simulación completó las diez rondas correctas y también detectó la pulsación incorrecta introducida durante la tercera ronda. En ese momento el juego mostró EE, apagó los LED y volvió a iniciar la preparación una vez liberado el botón.

Las mediciones obtenidas fueron 269, 353, 482, 196, 625, 311, 793, 426, 168 y 538 ciclos. Al finalizar, el

promedio mostrado fue 02 y los cuatro LED quedaron apagados.

El testbench también registró cero accesos a la RAM de datos durante toda la partida, lo que coincide con el análisis estático del programa realizado en la sección 2.1.

El resultado de la simulación fue PASS (errors=0). Al comparar los registros de salida y las mediciones entre RTL y netlist, ambos generaron los mismos 47 eventos. Por lo tanto, la implementación post-síntesis reprodujo el comportamiento observado en el RTL ciclo a ciclo.

La diferencia de 15 a 21 ciclos entre el instante programado para presionar el botón y el tiempo finalmente medido se explica por la latencia del sincronizador de entradas y por el bucle de sondeo que utiliza el programa para detectar las pulsaciones.

Resultado de la estrategia FF

La misma prueba se ejecutó utilizando la memoria unificada implementada con flip-flops.

En este caso también se obtuvo PASS tanto en la simulación RTL como en la simulación post-síntesis. La comparación entre ambas generó los mismos 47 eventos, y las mediciones de reacción coincidieron con las obtenidas mediante la estrategia ROM.

Esto permite comprobar que las dos arquitecturas mantienen el comportamiento funcional requerido por el juego y que la síntesis conserva ese comportamiento en el netlist.

Finalmente, es importante recordar que los valores mostrados en los displays durante estas pruebas utilizan la escala temporal reducida de simulación. Por ejemplo, una respuesta medida en 269 ciclos equivale a 1,345 décimas, considerando que cada décima corresponde a 200 ciclos. Como el sobrante no alcanza el umbral de redondeo de 100 ciclos, el resultado que aparece en el display es 01.

8. Reproducibilidad

Para que el trabajo pueda ser revisado y ejecutado nuevamente, dejamos en el README las instrucciones necesarias para instalar el entorno, generar el programa y ejecutar las corridas de LibreLane.

También se registraron las versiones de las herramientas y de la tecnología utilizadas, de manera que los resultados puedan relacionarse con una configuración específica y no dependan únicamente del computador donde se ejecutó el proyecto.

La implementación se realizó utilizando LibreLane 3.0.17, la revisión de GF180MCU identificada por el commit 54435919abffb937387ec956209f9cf5fd2dfbee y la biblioteca estándar gf180mcu_fd_sc_mcu7t5v0.

Los comandos de ejecución y generación del programa se encuentran documentados en el README, junto con las configuraciones correspondientes a las alternativas ROM y FF.

De esta manera, se conserva no solo el resultado final, sino también la información necesaria para reproducir el flujo y comprender las decisiones que se tomaron durante el desarrollo.